

CSI Motion Detection

Reverse-Engineering Learning Guide

Modules 1-11 - grounded in the actual code and the real, already-verified results in this repository's docs/PROJECT_HISTORY.md. Every number in this document came from an actual run of this project's own scripts.

Table of Contents

Table of Contents	2
Module 1 - High-Level Architecture	4
How Wi-Fi becomes a motion detector	4
End-to-end data flow	4
Why every stage exists (nothing here is incidental)	5
Module 2 - Repository Structure	6
Why <code>csi_common.py</code> is the single most important file to understand first	6
Module 3 - CSI Fundamentals (the physics)	7
OFDM and subcarriers	7
Amplitude, phase, and what this project actually captures	7
RSSI	7
Multipath, reflection, fading	7
The physical explanation for why motion detection works at all	8
Known physical limitation: the off-axis blind spot	8
Module 4 - Data Collection	9
The chain, concretely	9
Why 10Hz specifically	9
Why packet loss matters	9
The labeling protocol - the actual hard problem in data collection	9
Why per-session identity (<code>session_id</code>) matters	9
Module 5 - Dataset Generation	11
Step by step	11
Why windows straddling a label change are dropped, not truncated or kept	11
Real numbers from this exact pipeline (10-session dataset, 0.75s window)	11
Module 6 - Feature Engineering (266 features, and why each exists)	13
Group 1+2: per-subcarrier features ($128 + 128 = 256$ features)	13
Group 3: motion energy (2 features)	13
Group 4: RSSI (2 features)	13
Group 5: order-invariant summaries (6 features) - the cross-room generalization fix	13

Total feature count, verified	14
Module 7 - Machine Learning: Random Forest	15
The theory, briefly	15
How this project actually configures it	15
Feature importance	15
Why Random Forest, not a neural network (e.g. a CNN)	15
Module 8 - Evaluation	17
Leave-One-Session-Out (LOSO) - why not a random train/test split	17
Confusion matrix and the standard metrics, with this project's real numbers	17
ROC / AUC	17
The depth sweep, and why 100% training accuracy is fine here	17
The full accuracy story, in order (every number independently verified)	18
Module 9 - Live Prediction	19
The live pipeline, exactly as <code>csi_live_predict.py</code> / <code>csi_live_server.py</code> implement it	19
Two real bugs this project found live (invisible in short offline test sessions)	19
The manual escape hatch	19
Module 10 - Dashboard	20
Architecture: why a Python backend is still needed for a 'browser' dashboard	20
The WebSocket message protocol	20
Canvas waterfall	20
SVG energy sparkline and its hover tooltip	20
A subtle bug this project actually hit and fixed: a frozen dashboard that still says 'Connected'	20
Theme system	21
Module 11 - Improvements (already made, and what's still open)	22
Improvements already made this session, as real, verified engineering work	22
What's still genuinely open (from docs/PROJECT_HISTORY.md's own open-items list)	22

Module 1 - High-Level Architecture

The problem this project solves: tell "someone is moving in this room" apart from "the room is empty," using nothing but a single ESP32-S3 microcontroller and the existing Wi-Fi network already in the room - no camera, no wearable, no motion sensor, nothing the person has to carry or that captures identifiable imagery.

How Wi-Fi becomes a motion detector

Every Wi-Fi receiver has to know exactly how the radio channel distorted the signal on its way from the transmitter, or it cannot decode the data - this measurement is called **Channel State Information (CSI)**, and every 802.11n/ac chipset (including the ESP32-S3's) computes it internally as a normal part of receiving a packet. Normally CSI is thrown away the instant it's used for demodulation. This project's entire premise is that CSI is a free byproduct that happens to also encode something else useful: the physical shape of the room. A radio wave from the router to the ESP32 doesn't travel in one straight line - it also bounces off walls, furniture, and people, and all of those reflected copies arrive at slightly different times and phases and sum together at the receiver (this summation is called **multipath**). CSI captures the net effect of that summation, per subcarrier. A person walking through the room changes which surfaces the signal bounces off and how, which measurably changes CSI - even though nothing about the Wi-Fi data itself changed. That's the whole trick: motion is read off a side effect of ordinary Wi-Fi reception, not off a dedicated sensor.

End-to-end data flow

```
Person walks
  |
  v
Wi-Fi radio waves bouncing between router <-> ESP32 change shape
  |
  v
ESP32-S3 firmware (firmware/main/csi_node.c):
- pings the router every 100ms to force a steady stream of CSI updates
- esp_wifi's CSI callback fires per received frame, handing raw CSI to a
  FreeRTOS queue (kept fast: no heavy work in the radio callback itself)
- a separate task converts each subcarrier's raw (imag, real) pair to an
  amplitude via  $\sqrt{\text{imag}^2 + \text{real}^2}$ , formats one CSV-like line, and
  writes it to UART
  |
  v (serial cable, ~10 lines/sec, format: CSI_AMP,ts,label,n,rssi,amp0,amp1,...)
Python process (either the offline collector or a live tool) reads the
serial port and parses each line with csi_common.parse_line()
  |
  v
Frames are buffered into a fixed-size, non-overlapping time WINDOW
(0.75s = 8 frames at the deployed configuration)
  |
  v
Each window is turned into 266 CALIBRATED features (csi_common.calibrate_features) -
every value expressed relative to a live "empty room" baseline, not as an
absolute number
  |
  v
A pre-trained Random Forest (csi_model.joblib) classifies the window as
EMPTY (label 0) or MOVING (label 2)
  |
  v
The prediction is smoothed (majority of the last 5 window predictions) and
shown: Matplotlib window (csi_live_predict.py) or browser dashboard
(csi_live_server.py + csi_dashboard.html over a WebSocket)
```

Why every stage exists (nothing here is incidental)

- **Ping every 100ms** - CSI is only produced when a frame is actually received; without artificial traffic, CSI would arrive irregularly (whenever the router happens to send something), making fixed-rate windowing impossible. Pinging guarantees ~10Hz.
- **Separate FreeRTOS printer task, not printing inside the Wi-Fi callback** - an earlier version formatted and wrote to UART directly inside the CSI callback, which stalled the radio and dropped the frame rate from 10Hz to 5Hz. Moving that work off the radio's own task (handoff via a queue, callback does only a fast memcpy) fixed it.
- **BSSID filtering** - the ESP32 sees CSI for every Wi-Fi frame in range, not just the target router's. Frames from a neighbor's AP would contaminate the signal with an entirely different, unrelated channel. The firmware records the connected AP's BSSID on association and silently drops CSI from anything else.
- **Fixed-size windows, not raw per-frame prediction** - a single CSI frame is noisy; aggregating statistics (mean, std, frame-to-frame energy) over a short window is what makes "is this window moving or empty" a stable, learnable signal in the first place.
- **Calibration baseline, not absolute thresholds** - a fixed amplitude threshold learned on one day, in one room, with the hardware in one position, does not survive a different day, a repositioned router, or a different room. Comparing every window against a *live, recent* "this room, right now, confirmed empty" baseline is what makes the same trained model usable across sessions and (with real limits - see Module 6) across rooms.
- **Prediction smoothing** - even a good classifier occasionally misclassifies one noisy window. Requiring a majority of the last 5 raw predictions to agree before the *displayed* state changes stops single-window noise from flipping the UI.

Self-check for this module: without looking at any code, try to redraw the arrow diagram above from memory, and for each arrow, say in one sentence why that stage can't be skipped. If you can't justify a stage, that's exactly the thing to ask about next - this is meant to be checked interactively, not just read.

Module 2 - Repository Structure

Every folder and file exists to serve exactly one stage of the Module 1 pipeline. The table below is organized that way on purpose - group by pipeline stage, not alphabetically - so the "why does this exist" question has an obvious answer.

Path	Pipeline stage	Depends on / feeds	If removed...
firmware/main/csi_node.c	Sensing (hardware)	Nothing upstream; everything else depends on this	Project stops at the point of sensing
firmware/main/wifi_secrets.h (gitignored)	Sensing (config)	Read by csi_node.c at build time	Firmware can't connect to Wi-Fi; wifi_secrets.h.example
csi_label_collector.py	Data collection	Reads firmware's serial stream; writes to part_N_data/	Is only how we start new labeled sessions
csi_live_monitor.py	Data collection (sanity check)	Reads serial stream; no model	Lose the pre-model waterfall/energy sanity check (S)
part_1_data .. part_8_data/ room2_part_1-2_data/	Data (the dataset itself)	Produced by the collector; consumed by training	No training/retraining; downstream can run
csi_common.py	Shared core logic	Imported by every other Python file in the project	Every training AND live-inference script breaks; this is the only file that can't be removed
train_model.py	Model training	csi_common.py + part_N_data/; produces model.joblib	No reason to retrain a model
evaluate_holdout.py	Model validation	train_model.build_dataset(); no model	File needed to test this new session usable 'true-generalization'
compare_models.py	Model validation	Same features as train_model.py, 6 different classifiers	Differs the classifiers that Random Forest is actually the best
model_evaluation.py, full_model_report.py	Model validation / reporting	Trained-model metrics -> PDF report	Lose the automated health-check reports (confusion matrix)
csi_model.joblib	Model artifact	Produced by train_model.py; loaded by live tools	Big dependency: model - live prediction can't start
csi_live_predict.py	Live inference (desktop)	csi_common.py + csi_model.joblib + SerialPort	SerialPort + Matplotlib live view
csi_live_server.py	Live inference (backend)	csi_common.py + csi_model.joblib + SerialPort	SerialPort + Web Socket connect to
csi_dashboard.html	Live inference (frontend)	Connects to csi_live_server.py over Web Socket	Web Socket + Python Matplotlib + JS works independently
report/*.tex	Documentation	Hand-written, references numbers from PROJECT_HISTORY and	Loss of PROJECT_HISTORY and
docs/PROJECT_HISTORY.md	Documentation	Nothing; this is the ground truth for every decision	Lose decision history - every 'why' in this
README.md, requirements.txt, .gitignore, LICENSE	Repo hygiene	N/A	Repo becomes harder to onboard into / less safe to

Why csi_common.py is the single most important file to understand first

It is the only file imported by every other Python file in the project - both training scripts and both live-inference tools. This is a deliberate architectural choice: if feature extraction or calibration math lived separately in the training path and the live path, they could silently drift apart (a bug fixed in one copy but not the other), and a model trained on one feature definition would be scored live against a subtly different one with no error message - just quietly wrong predictions. Centralizing it means offline and online math are provably identical by construction.

Module 3 - CSI Fundamentals (the physics)

OFDM and subcarriers

802.11n/ac Wi-Fi (what the ESP32-S3 uses here) transmits using **OFDM** (Orthogonal Frequency-Division Multiplexing): instead of sending one signal on one frequency, the channel is split into many narrow parallel sub-channels called **subcarriers**, each carrying a fraction of the data simultaneously. This project's 20MHz channel produces **128 subcarriers** per CSI reading (confirmed directly from the deployed model's metadata and every recorded session's data). Splitting the channel this way is what makes CSI useful for sensing at all: instead of one number describing "how disturbed is the channel," you get 128 numbers, one per narrow frequency slice, and different physical obstructions/reflections affect different subcarriers differently (this is exactly why per-subcarrier features carry more information than a single aggregate number - see Module 6).

Amplitude, phase, and what this project actually captures

For each subcarrier, the receiver computes a complex number (a real and an imaginary component) describing how that subcarrier's signal was reshaped. Two things can be derived from that complex number: **amplitude** (how strong the signal is on that subcarrier - magnitude) and **phase** (how much its timing/waveform was shifted). The ESP32-S3's CSI callback hands the firmware a raw buffer of (imag, real) int8 pairs per subcarrier. Looking directly at `csi_node.c`:

```
int8_t imag = item.buf[i * 2];
int8_t real = item.buf[i * 2 + 1];
int amp = (int)lroundf(sqrtf((float)(imag * imag + real * real)));
```

Important, and directly relevant to "can this detect position": phase IS present in the raw buffer the ESP32 receives, but the firmware computes amplitude and *discards phase before the data ever leaves the device*. Every feature, every model, every dashboard number in this whole project is amplitude-only. Real angle-of-arrival position tracking needs phase (plus multiple synchronized antennas), which would require a firmware change to transmit phase at all, not just a software change downstream.

RSSI

RSSI (Received Signal Strength Indicator) is a single number per received frame describing overall signal strength, independent of the per-subcarrier CSI detail. The firmware reads it straight from the driver (`info->rx_ctrl.rssi`) and streams it alongside the 128 amplitudes. It's coarser than CSI (one number vs. 128) but still useful: RSSI dropping noticeably during a window is a real, independent signal that something is absorbing/blocking more of the signal path - it turned out to be the single most important individual feature in this project's earliest feature-importance analysis.

Multipath, reflection, fading

- **Multipath:** the signal from router to ESP32 doesn't travel one path - it also bounces off walls, furniture, and people, and all those delayed copies sum together at the receiver. This sum is what CSI measures.
- **Reflection:** each bounce off a surface changes the wave's phase and attenuates its amplitude depending on the material. A person's body is a large, lossy, constantly-moving reflector - so their movement visibly perturbs the summed signal.

- **Fading:** because multipath copies can interfere constructively or destructively at a given frequency, some subcarriers naturally read stronger and some weaker even in a static, empty room - this is why *absolute* per-subcarrier amplitude is not a usable feature on its own (Module 6 explains the fix: calibrate against a live baseline rather than a fixed number).

The physical explanation for why motion detection works at all

A stationary empty room has a fixed multipath pattern - the same walls and furniture in the same place produce the same interference pattern, session to session, modulo slow drift (thermal, minor environment changes). A person walking through the room is a large, moving reflector: as they move, the multipath pattern is disturbed frame to frame - not just "different," but *changing*. That's why this project's single most important derived feature is **motion energy**: the mean frame-to-frame amplitude change, averaged across subcarriers (`np.abs(np.diff(amps, axis=0)).mean()`). A static room, however its raw amplitudes happen to sit, has low motion energy; a moving person spikes it, because the pattern itself is changing, not merely offset from some fixed value.

Known physical limitation: the off-axis blind spot

Single-link CSI sensitivity is strongest for movement near the direct line between the ESP32 and the router (the strongest, most direct multipath component is the most easily disturbed). Movement more than roughly 1m off that direct line is detected much more weakly. This is genuine Fresnel-zone physics, not a bug, and not fixable by more data or model tuning - the two real fixes are repositioning the hardware to diagonally opposite corners of the room (maximizes the direct-path coverage of the space) or adding a second sensing node at a different position (Module 11 discusses this).

Module 4 - Data Collection

The chain, concretely

```
ESP32 (wifi_csi_rx_cb, runs in the Wi-Fi task - must be FAST)
-> non-blocking xQueueSend to csi_queue (drops + counts if full, never blocks)
-> csi_print_task (separate FreeRTOS task, does all the formatting + UART I/O)
    -> UART line: "CSI_AMP,<ts_us>,<label>,<num_subcarriers>,<rssi>,<amp0>,...,<amp127>"
        -> csi_label_collector.py reads the serial port, parses, writes CSVs
```

Why 10Hz specifically

The firmware pings the router every 100ms (`interval_ms = 100` in `start_ping()`), and every ping reply (plus incidental beacon traffic) triggers a fresh CSI callback - so the achieved rate is roughly 10 frames/second, confirmed by `FRAME_HZ = 10.0` in `csi_common.py` and by the frame counts in every recorded session (~10 rows/second of wall-clock recording time). This rate is a deliberate trade: fast enough that a 0.75s window still contains several frames (8, at 10Hz) for the mean/std statistics to be meaningful, slow enough not to overwhelm the UART link or the printer task. It is not an inherent Wi-Fi limit - it's set by how often the ESP32 chooses to ping.

Why packet loss matters

The CSI callback runs inside the Wi-Fi driver's own task; if it did any slow work (string formatting, UART writes), it would stall packet reception itself - this actually happened in an earlier version of this firmware and cut the frame rate from 10Hz to 5Hz. The fix was moving all formatting/I/O to a separate task and handing data off through a fixed-size FreeRTOS queue (`CSI_QUEUE_LEN = 32`). If the printer task can't keep up with the queue (e.g. the UART baud rate is a bottleneck), `xQueueSend` fails non-blockingly and the frame is dropped and counted (`frames_dropped`), rather than blocking the radio and cascading the problem. Occasional dropped frames just thin out one window's frame count slightly; a systemic drop (UART baud too low for the data rate) would show up as a persistent `frames_dropped` warning and a visibly slower effective frame rate.

The labeling protocol - the actual hard problem in data collection

You must be physically absent from the room to record a clean "empty" example, and physically present (and moving) to record a "moving" one - a person can't record both labels at the same instant.

`csi_label_collector.py`'s state machine solves this with an auto-cycling protocol:

```
LEAVE (5s countdown, frames tagged settling=1, excluded from training)
-> EMPTY (auto-recorded for 60s once the countdown ends)
    -> WAIT (idle, not recording; operator walks back in)
        -> press 'm' -> MOVING (auto-recorded for 60s)
            -> LEAVE again (auto), repeats
```

The `settling=1` flag on leave-window frames exists because a person lingering near the doorway while leaving is still a real disturbance - training on those frames as "empty" would poison the empty class with borderline-occupied data. `train_model.py`'s `load_session()` drops every `settling==1` row before anything else happens.

Why per-session identity (`session_id`) matters

Every recording gets a unique session_id (timestamp + random suffix), used as the LOSO grouping key in Module 8. This is what lets the evaluation methodology honestly ask "does this generalize to a recording session the model has never seen," instead of accidentally testing on frames from the same session it trained on.

Module 5 - Dataset Generation

This module follows one CSI packet from a raw CSV row all the way to one row in the training matrix, exactly as `train_model.py`'s `build_dataset()` actually does it.

Step by step

- **Load** (`load_session()`): read `all_csi_data.csv`, drop every row with `settling==1`, sort by timestamp.
- **Find empty-block starts** (`find_empty_block_starts()`): every index where a LABEL 0 (empty) row immediately follows a different label (or is the very first row) - these are the moments a fresh calibration baseline should be computed.
- **Compute per-block baselines** (`compute_block_baselines()`): for each empty-block start, take its own first `calib_seconds` (10s = 100 frames) and run `compute_baseline()` on just that slice.
- **Window** (`windowize()`): slide a fixed-size, non-overlapping window (8 frames at the deployed 0.75s setting) across the session. A window is **dropped** if the frames inside it don't all share the same label (it would straddle a real empty<->moving transition and be ambiguous to train on).
- **Score against the governing baseline**: each kept window is compared against whichever empty block's baseline most recently preceded it - not always the session's first baseline. This is what lets the pipeline absorb a mid-session change (the AC turning on between two empty blocks, for example) instead of comparing everything against a single stale snapshot.
- **Featurize** (`raw_window_stats()` + `calibrate_features()`): turn the window into one 266-value feature vector (Module 6 covers every value).
- **Collect**: append the feature vector, its label, and its source session name (the LOSO grouping key) into the growing `X / y / groups` arrays.

Why windows straddling a label change are dropped, not truncated or kept

A window that spans the exact instant a person starts or stops moving is genuinely ambiguous - it contains real information from both classes and no ground truth for which one the aggregate statistics "belong to." Keeping it (with either label) would inject label noise; the pipeline simply discards it. This costs a small amount of data at every label transition but keeps every remaining training example unambiguous.

Real numbers from this exact pipeline (10-session dataset, 0.75s window)

Session	Raw frames	Windows kept	Empty blocks (recalibration points)
part_1_data	1,200	150	1
part_2_data	2,400	300	2
part_3_data	2,443	305	3
part_4_data	2,400	300	2
part_5_data	2,400	300	2
part_6_data	2,401	297	2
part_7_data	2,400	300	2

Session	Raw frames	Windows kept	Empty blocks (recalibration points)
part_8_data	2,400	300	2
room2_part_1_data	4,801	600	4
room2_part_2_data	3,600	450	3

Total: 3,302 windows, 266 features each, near-perfectly balanced classes (1,654 empty / 1,648 moving) - this exact table is what `train_model.py` printed when the 10-session model was last trained.

Module 6 - Feature Engineering (266 features, and why each exists)

Every feature is computed relative to a calibration baseline - nothing here is an absolute number. The guiding question for this whole module isn't "what does this function do" (that's obvious from the code) but **why compute this feature** - what specific failure would happen without it.

Group 1+2: per-subcarrier features (128 + 128 = 256 features)

Feature (x128 each)	Formula	Why it exists
<code><i>subcarrier</i>_mean_delta</code>	<code>window_mean - baseline_mean</code>	A delta (not the raw mean) cancels out whatever the baseline's absolute value is
<code><i>subcarrier</i>_std_ratio</code>	<code>window_std / max(baseline_std, floor)</code>	A ratio (not the raw std) means an already-noisy baseline (a fan, intercom, etc.) doesn't blow up the ratio

The dead-subcarrier bug, and why the floor exists: ~20 of the 128 subcarriers are structurally dead (OFDM guard bands / DC nulls) - always exactly zero amplitude, so their baseline std is exactly 0.0. Dividing by ~zero means even trivial live noise (std=0.01) on one of them produces a ratio of ~10,000 instead of a healthy ~1.0. This was a real bug found live (invisible in short offline sessions, where it rarely triggered by chance) - it caused the live dashboard to predict MOVING almost constantly. The fix, `amp_std_floor`: no subcarrier's baseline std counts as smaller than 10% of that baseline's own median std, computed once per baseline in `compute_baseline()`.

Group 3: motion energy (2 features)

`energy_mean_ratio`, `energy_std_ratio` - the mean/std of frame-to-frame amplitude change within the window, each expressed as a ratio against the baseline's own energy. This is the single feature most directly tied to the physical explanation in Module 3: a static room has low, flat energy; movement spikes it. It is not always the deciding feature, though - one recorded session (part_8) showed this aggregate number stay completely flat between empty and moving (0.98 vs 0.97) while per-subcarrier features still separated the classes cleanly - motivating Group 5 below.

Group 4: RSSI (2 features)

`rss_i_mean_delta`, `rss_i_std_ratio` - same delta/ratio treatment as the per-subcarrier features, applied to the single overall signal-strength number.

Group 5: order-invariant summaries (6 features) - the cross-room generalization fix

Feature	What it measures
<code>std_ratio_max</code>	The single most-disturbed subcarrier's ratio, wherever it is
<code>std_ratio_topK_mean (K=10)</code>	Mean ratio of the 10 most-disturbed subcarriers
<code>std_ratio_p90</code>	90th percentile of all 128 ratios - robust to one outlier
<code>std_ratio_frac_elevated</code>	Fraction of subcarriers with ratio > 1.5
<code>mean_delta_absmax</code>	The single largest <code> amp_mean_delta </code> , wherever it is
<code>mean_delta_absmax_topK_mean</code>	Mean of the 10 largest <code> amp_mean_delta </code> values

Group 1/2's per-subcarrier features are tied to *specific indices* - a Random Forest trained on one room can end up splitting on 'index 43 changed,' which is only meaningful because index 43 happens to be where that

particular room's multipath geometry puts the sensitive spot. A different room's geometry puts that sensitivity somewhere else entirely, and index-tied features have no way to transfer the concept. Group 5 answers 'is *something* in the band disturbed' without caring which index - added alongside the original 260 features (not a replacement), retrained with no LOSO regression (95.87% -> 95.54%, within normal noise), and the live cross-room test later confirmed the underlying idea is real: it moved detection from an informal '6/10' guess to a measured 82.5-98.4% depending on the session (Module 8 has the full numbers).

Total feature count, verified

128 (mean_delta) + 128 (std_ratio) + 2 (energy) + 2 (RSSI) + 6 (order-invariant) = **266** - matches csi_model.joblib's own stored metadata (len(feature_names) == 266) exactly.

Module 7 - Machine Learning: Random Forest

The theory, briefly

- **Decision tree:** a tree of if/else splits on feature values, ending in a leaf that predicts a class. A single tree grown without limits will fit the training data almost perfectly, including its noise (overfitting).
- **Bootstrap:** sampling *with replacement* from the training set to build a new, slightly different training set of the same size - some original rows appear multiple times, others not at all.
- **Bagging** (bootstrap aggregating): train many trees, each on its own bootstrap sample, then combine their predictions. Averaging many trees that each overfit *differently* (because each saw a different bootstrap sample) cancels out much of that overfitting.
- **Random Forest** adds one more layer of randomness on top of bagging: at every split, each tree only considers a random subset of the available features, not all of them. This decorrelates the trees further - without it, if one feature is very strong, almost every tree would split on it first and the trees would end up highly similar, which weakens the averaging benefit.
- **Voting:** for classification, the forest's prediction is the majority vote across all trees (`clf.predict()`); `predict_proba()` instead returns the fraction of trees that voted for each class, used as a confidence score.

How this project actually configures it

```
RandomForestClassifier(n_estimators=200, random_state=42)
# max_depth left at the default (None) -> trees grow until leaves are pure
```

`n_estimators=200` trees. `random_state=42` makes results exactly reproducible run to run (confirmed directly in this project: re-running `train_model.py` on the same 10 sessions reproduced the identical 95.07% LOSO accuracy and identical per-fold numbers). `max_depth` is unrestricted by default - Module 8 explains why that's fine here specifically, not just assumed safe.

Feature importance

A Random Forest can report, per feature, how much including it reduced impurity (class-mixing) across all its splits in all its trees, averaged. This project's own feature-importance runs consistently `surface_rssi_std_ratio` as the single most important feature, followed by a cluster of per-subcarrier std-ratio features concentrated around indices ~40-48 and ~92-108 - the physical explanation (Module 3) is that this is where the room's multipath geometry happens to put the most sensitive band to disturbance. That same cluster's room-specificity is exactly what motivated the order-invariant features in Module 6.

Why Random Forest, not a neural network (e.g. a CNN)

This wasn't assumed - `compare_models.py` ran the identical LOSO methodology on 6 model families using identical calibrated features: Random Forest, Gradient Boosting, Logistic Regression, SVM (RBF), SVM (linear), and KNN. Random Forest won outright: 95.14% mean accuracy with the *lowest* variance across folds (std 0.025) - not just the highest number, but the most consistent, meaning it wasn't winning by getting lucky on a couple of easy sessions. Gradient Boosting was a close second (94.71%). A CNN specifically was never tested, for a concrete reason: with only 266 hand-engineered, already-informative features and a few thousand

labeled windows total, there isn't enough data for a deep network to learn useful representations from scratch that would beat features an expert already designed around the physics - CNNs earn their advantage on raw, high-dimensional, unstructured input (like images) where hand-engineering isn't feasible; here the hand-engineered features already are the useful representation.

Module 8 - Evaluation

Leave-One-Session-Out (LOSO) - why not a random train/test split

Windows within one recording session are highly correlated: same room, same day, same hardware position, same person's specific movement style. A random split would let windows from the same session appear in both train and test, leaking session-specific quirks across the split and inflating the reported accuracy dishonestly - the model would look better than it actually generalizes. LOSO instead holds out one *entire* session per fold: a fresh model trains on every other session and is scored only on the one it has never seen at all. This is implemented directly via `sklearn.model_selection.LeaveOneGroupOut`, grouped by `session_id`.

Confusion matrix and the standard metrics, with this project's real numbers

From the most recent independent evaluation run (`model_evaluation.py`, 10 sessions, out-of-fold predictions - every prediction came from a fold that never trained on that window's session):

Metric	Value
Out-of-fold accuracy	94.2% (1,547/1,654 empty correct, 1,565/1,648 moving correct)
False alarms (empty called moving)	107
Missed detections (moving called empty)	83
Mean train accuracy across folds	100.0%
Mean held-out accuracy across folds	95.07%
Mean train/held-out gap	4.9-4.93 points

Precision = of everything predicted MOVING, what fraction really was moving (false-alarm control). **Recall** = of everything that really was moving, what fraction got caught (miss control). **F1** is their harmonic mean. This project reports all three both per-class and as macro/weighted averages, plus MCC and Cohen's kappa (chance-corrected agreement metrics, useful because they don't get inflated by a large majority class the way raw accuracy can - though this dataset is nearly perfectly class-balanced, so the difference from raw accuracy here is small).

ROC / AUC

Binary-classification-only: sweeps the decision threshold and plots true-positive rate against false-positive rate at every threshold; AUC (area under that curve) summarizes how well the model ranks moving-windows above empty-windows across *all* possible thresholds, not just the default 50% one. `full_model_report.py`'s run on this project's real data measured **ROC AUC = 0.986** - very close to a perfect 1.0, meaning the model's confidence scores are genuinely well-ordered, not just its hard predictions.

The depth sweep, and why 100% training accuracy is fine here

An unconstrained Random Forest grows every tree until its leaves are pure - this *structurally guarantees* 100% training accuracy and is not, by itself, evidence of a problem. What actually matters is the held-out accuracy and the gap between the two. This project verified that directly with a validation curve (sweeping `max_depth` from 2 up to unlimited):

max_depth	Train accuracy	Held-out (LOSO) accuracy
2	92.0%	90.9%
3	94.6%	92.3%
5	97.1%	93.7%
8	98.9%	94.5%
12	99.6%	94.6%
20	100.0%	94.8%
None (unlimited)	100.0%	95.1%

If 100% training accuracy were genuinely hurting generalization, restricting depth should have *improved* held-out accuracy as train accuracy dropped away from 100% - instead held-out accuracy *keeps rising* all the way to the unrestricted depth. That rising-together pattern (not a falling-apart one) is the actual evidence this specific model isn't overfitting, not an assumption taken on faith.

The full accuracy story, in order (every number independently verified)

Milestone	Result
3-session baseline (2.0s window, absolute features)	96.96% LOSO
4-session, calibration added (7a/7b)	97.30% LOSO / 95.83% true held-out
4-session, per-block recalibration added (Module 8's own fix, section 8)	98.34% LOSO
8-session, 0.75s window, absolute+calibrated	95.87% LOSO
8-session, + order-invariant features (Module 6, group 5)	95.54% LOSO (no regression)
Cross-room held-out (train 8 room-1 sessions, test 2 new room-2 sessions)	82.5% / 95.3% (88.0% combined)
10-session retrain (both rooms folded in)	95.07% LOSO / 94.2% out-of-fold
Cross-room folds within the 10-session LOSO	room2_part_1: 84.7% room2_part_2: 98.4%

The wide swing between the two room2 sessions (82.5%/84.7% vs. 95.3%/98.4%) traces to a real physical cause, not model instability: the first room2 recording's movement stayed mostly off the direct ESP32-router line (flat motion energy, 0.979 vs 0.992), while the second recording's movement crossed it directly (strong separation, 1.218 vs 3.223) - the off-axis blind spot from Module 3, observed and measured, not guessed at.

Module 9 - Live Prediction

The live pipeline, exactly as `csi_live_predict.py` / `csi_live_server.py` implement it

Serial bytes arrive continuously

- > `parse_line()` extracts (rssi, amps) per line, ignores anything that doesn't match the `CSI_AMP,...` format
- > CALIBRATION PHASE (first `calib_seconds` after data starts):
 - buffer confirmed-empty frames -> `compute_baseline()` once buffered enough (this is the ONE moment you must actually leave the room)
- > live frames appended to a rolling deque(`maxlen=window_frames`)
- > once the deque is full, EVERY new frame recomputes `make_features()` on the whole window and re-predicts (not just once per window boundary - the window slides by one frame at a time, giving a fresh prediction ~10x/sec)
- > `clf.predict()/predict_proba()`
 - > `PredictionSmoother.update()`: majority of last 5 raw predictions -> the DISPLAYED (confirmed) state
 - > `RollingCalibrator.observe()`: if confirmed EMPTY for a full `calib_frames` stretch, quietly blend in a fresh baseline sample
 - > shown on screen / broadcast to the dashboard

Two real bugs this project found live (invisible in short offline test sessions)

- **Dead-subcarrier ratio blowup** (Module 6's `amp_std_floor`) - the order-invariant features hunting for the single MAX std ratio across all 128 subcarriers kept getting hijacked by trivial noise on a structurally-dead subcarrier, producing ratios of ~10,000 on nearly every window. Fixed with a variance floor relative to the baseline's own median spread.
- **Rolling recalibration drift** - the first version of `RollingCalibrator` fully *replaced* the baseline every recalibration using only that fresh ~10s sample. A short sample's noise estimate is itself noisy and can land smaller than the room's true typical noise purely by chance; with nothing pulling it back, repeated full-replacement recalibration could drift toward an unrealistically tight baseline over a longer session, making ordinary conditions start reading as motion. Fixed with two mechanisms: **blend** (each recalibration only nudges the baseline `blend_alpha=0.3` toward the fresh sample, not replace it) and a **floor** (std values can never be blended down below `min_std_fraction=0.6` of what the original startup calibration measured - that startup calibration is a fixed, trustworthy reference nothing live is allowed to exceed in sensitivity).

The manual escape hatch

If the confirmed prediction ever locks onto MOVING for any reason, recalibration is gated behind being confirmed EMPTY first - so a genuinely stuck bad state had no way to self-correct. Rather than auto-detecting 'stuck' (risky - could suppress a real, long motion event), both live tools expose a manual override: press 'r' in the Matplotlib window, or the dashboard's 'Force recalibration now' button (sends `{"type": "recalibrate"}` to the server over the same WebSocket used for live data, in the reverse direction).

Module 10 - Dashboard

Architecture: why a Python backend is still needed for a 'browser' dashboard

A browser cannot open a Windows COM port directly (outside the Chrome-only, narrowly-supported Web Serial API), so `csi_live_server.py` exists purely to bridge serial -> WebSocket: it opens the port in a background thread (identical parsing/calibration/prediction logic to the Matplotlib tool, imported from the same `csi_common.py`), and broadcasts JSON messages over `ws://localhost:8765` to any connected browser tab. Re-implementing the trained Random Forest in JavaScript to go fully browser-native was considered and rejected: two implementations of the same prediction logic could silently diverge exactly the way `csi_common.py` was built to prevent.

The WebSocket message protocol

type	Direction	Carries
init	server -> client	subcarrier count (once, after the stream locks)
calibrating	server -> client	buffered/needed frame counts, remaining seconds
calibrated	server -> client	timestamp of the just-completed baseline
recalibrated	server -> client	timestamp of a rolling/forced recalibration
frame	server -> client	one waterfall row + rssi + energy + raw & confirmed prediction + confidence
error / warning	server -> client	serial/parse problems - cached and replayed to any client that connects after the fact
recalibrate	client -> server	manual force-recalibration request (the only client-to-server message)

Canvas waterfall

A `Float32Array` ring buffer (150 rows x active-subcarrier-count) is scrolled up by one row and the new row appended at the bottom on every frame message. Colors come from linearly interpolating each value's percentile position (2nd-98th percentile, computed once from the first ~30 calibration frames) between two fixed hex colors, written directly into an `ImageData` buffer and blitted with `putImageData` - this bypasses per-pixel DOM/canvas-path drawing calls entirely, which is what keeps it fast enough to redraw at ~10Hz.

SVG energy sparkline and its hover tooltip

The energy chart is a plain SVG path, regenerated (its whole `innerHTML` replaced) on every incoming frame. The hover tooltip is deliberately a plain absolutely-positioned HTML `<div>`, not an SVG element inside that same chart - if it were SVG, the `innerHTML` replacement on the very next frame would wipe out the tooltip mid-hover on a live-updating chart. The tooltip instead listens for `mousemove`, maps the cursor's x-position back to a data index using the geometry saved from the most recent draw, and positions itself in the DOM independent of the chart's own redraw cycle.

A subtle bug this project actually hit and fixed: a frozen dashboard that still says 'Connected'

If an incoming frame's row length doesn't match the waterfall's allocated width (e.g. the server restarted mid-session with a different subcarrier count), the original code threw inside the row-copy step - and because that ran before every other on-screen update in the frame handler, one bad frame silently broke *all future*

rendering while the WebSocket itself stayed open and green ('Connected'). A JavaScript exception inside a WebSocket's onmessage handler does not close the socket - it just fails silently. The fix: validate the row length before touching the canvas buffer, wrap the rest of the per-frame handling in try/catch so one bad frame can't wedge every future one, and add a live 'last frame received Xs ago' indicator so a genuine freeze is now visibly distinguishable from a quiet-but-healthy connection.

Theme system

Three states (Auto/Light/Dark), cycled by a button and persisted in localStorage. Implemented with CSS custom properties declared three times: a light default, a @media (prefers-color-scheme: dark) block scoped with :root:not([data-theme="light"]) (so OS-dark applies unless the user explicitly stamped light), and an explicit :root[data-theme="dark"] block that always wins regardless of OS setting. This is what lets the user's manual choice override the OS setting in either direction, not just follow it.

Module 11 - Improvements (already made, and what's still open)

Improvements already made this session, as real, verified engineering work

- **Cross-room data collection and retrain:** recorded two sessions in a genuinely different room, ran a true held-out evaluation (82.5%/95.3%), then folded both into training for a 10-session retrain (95.07% LOSO / 94.2% out-of-fold) - the model's first exposure to more than one room's geometry.
- **csi_live_server.py performance fix:** replaced two separate forest walks (predict() + predict_proba()) with a single predict_proba() call run in a thread executor, off the asyncio event loop entirely - this both halved per-frame inference cost and stopped WebSocket keepalive pings from being delayed by prediction latency (the actual cause of a 'keepalive ping timeout' disconnect that had been misread as a crash).
- **full_model_report.py:** a generic (not just this-project-specific), reusable LOSO evaluation-report generator - 16 requested metrics, ~20 diagnostic figures, an auto-paginated PDF with a real table of contents, built and actually run end-to-end against this project's real 10-session data as verification, not just written and assumed to work.
- **Dashboard redesign:** two passes - first a general professional-IoT-dashboard pass (status pills, hero prediction card, unified card system), then a second pass specifically addressing generic-template tells (flat instrumented background instead of a decorative gradient, monospace for all telemetry numbers, a state-tinted card edge and confidence ring on the prediction badge, merged waterfall+energy into one Signal card, demoted secondary/diagnostic cards).

What's still genuinely open (from docs/PROJECT_HISTORY.md's own open-items list)

- **A third, genuinely different room** - two rooms shows a real trend exists; a third is what would tell you whether accuracy keeps improving with more room diversity or plateaus.
- **A second person / different movement style** - every recorded 'moving' example so far is the same person's walking. Untested whether the model detects 'a person' in general or 'this person' specifically.
- **An extended, unattended live soak test** - the two live-only bugs in Module 9 were both found by exactly this kind of long-running real test; the just-shipped server performance fix hasn't been soak-tested yet either.
- **validate_session.py** - every new session so far has been manually diagnosed by hand (subcarrier consistency, motion-energy/RSSI separability vs. existing sessions). Proposed multiple times, never built as an actual automated script.
- **Feature redundancy** - the health check consistently flags dozens of highly correlated feature pairs among the top 25 by importance (neighboring subcarriers move together; the 6 order-invariant features are near-duplicates of each other by construction). Not urgent - Random Forest tolerates correlated features fine - but a real candidate for future trimming toward a leaner model.
- **A second ESP32 node** - the concrete next step discussed for actually moving beyond single-link CSI: it wouldn't give true (x,y) position (Module 3's phase limitation still applies), but it would directly fix the off-axis blind spot (a second node creates a second sensitive line at a different angle) and enable coarse zone-level

classification ("nearer node A vs. node B") - both real, achievable with the existing amplitude-only pipeline, unlike true triangulation.

A note on how this guide is meant to be used: this document deliberately does NOT include the interactive parts of the original learning plan - the mini-quizzes and 'ask me interview questions until I'm comfortable' loop. A static PDF can't do that; it can only hand you the facts to be quizzed on. Treat this as the reference material, and do the Socratic back-and-forth live in conversation, module by module - that's where the actual learning (and the honest 'I don't actually know this yet' signal) happens.